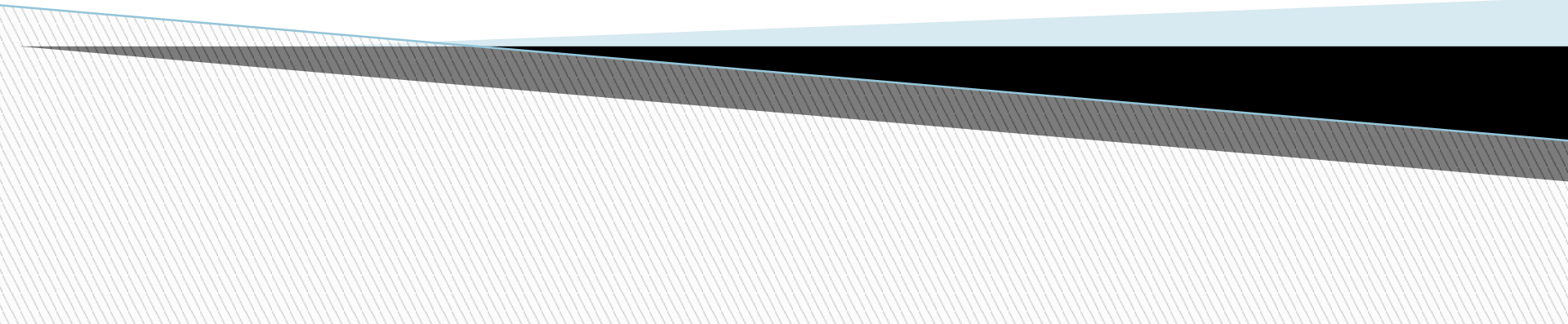


Hashing

Part(2)

Data structure



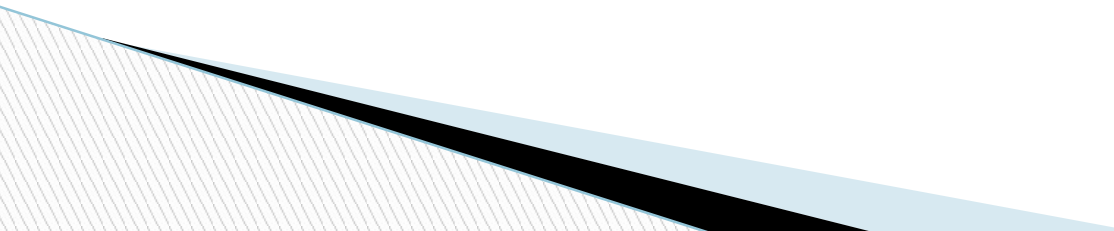
Collision Solutions

Solution #1: Search for an empty location

- ❑ Can stop searching when we find the value or an empty location.
- ❑ Search must be wrap-around at the end.

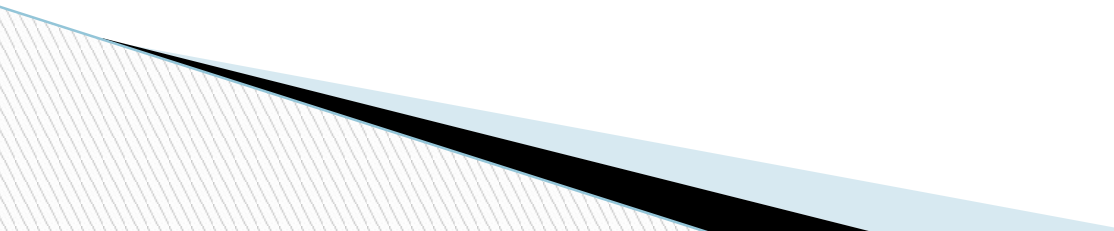
Solution #2: Use a second hash function and a third, and a fourth, and a fifth, ...

Solution #3: Use the array location as the header of a linked list of values that hash to this location.



Collision

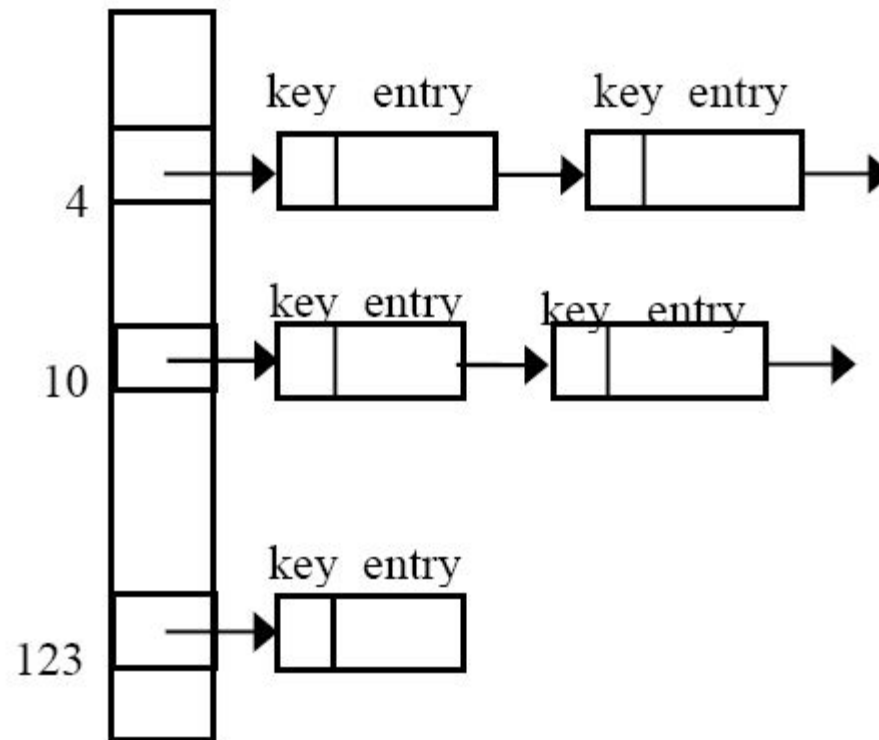
- When two values hash to the same array location, this is called a *collision*

 - There are mainly three methods to handle collision:
 - 1) **Separate Chaining**
 - 2) **Rehashing**
 - 3) **Open Addressing**
 - **Linear probing**
 - **Quadratic probing**
- 

Chaining

- Keys do not have to be stored in the table itself
- Each position of the table is associated with a linked list or *chain of structures*
 - **The table stores only pointers**
- This method is called *separate chaining/(Open Hashing)*,
- For short linked lists, this is a very fast method

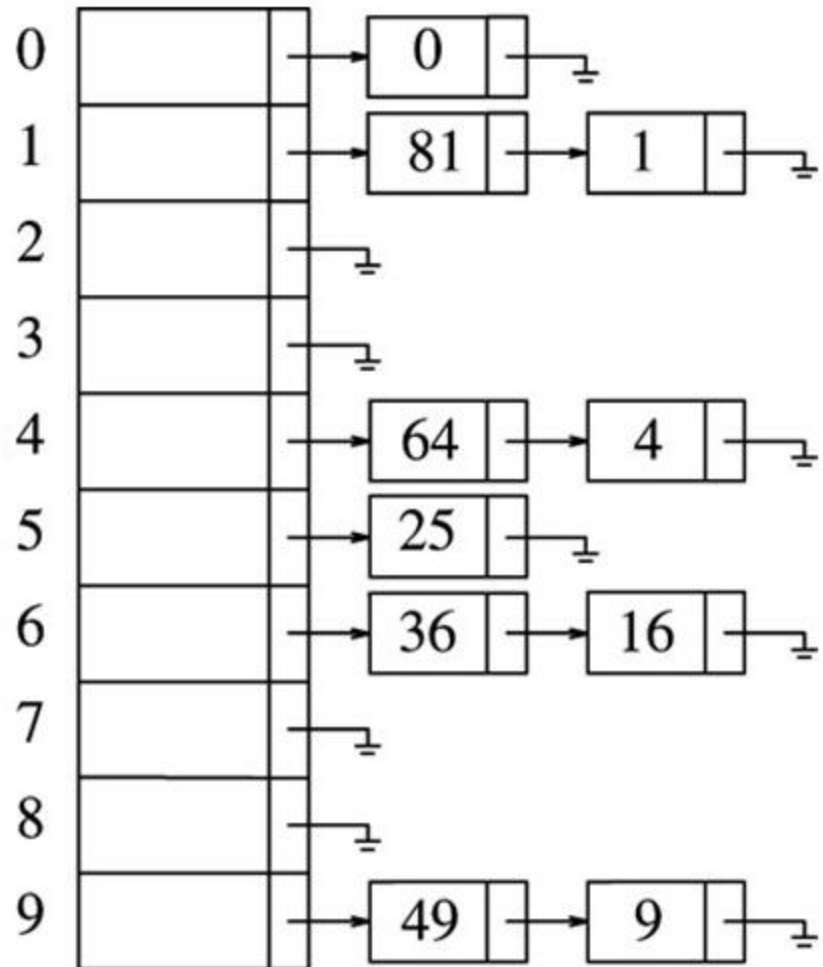
Separate Chaining



Separate Chaining

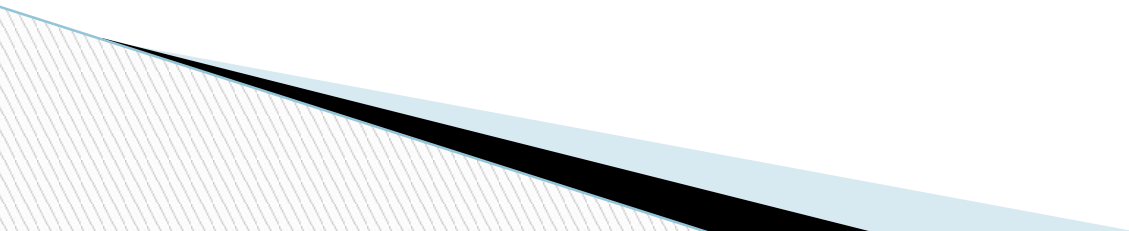
- Each table entry stores a list of items
- Multiple keys mapped to the same entry maintained by the list
- Example
 - $\text{Hash}(k) = k \bmod 10$
 - (10 is not a prime, just for illustration)

```
List* b= new List[10];  
b[81%10].insertAtTail(81); //b[1]  
b[1%10].insertAtTail(1); //b[1]  
b[36%10].insertAtTail(36); //b[6]
```



Advantage of link list

- Simpler insertion and removal
- Array size is not a limitation in separate chaining



Disadvantage of Separate Chaining

- ❑ Increasing the length of these lists can significantly degrade retrieval performance.
 - If the chain becomes long, then search time can become $O(n)$ in the worst case.
- ❑ Uses extra space for links.
 - The table stores only pointers, and each node requires one pointer field. Therefore, for n keys, $n + TSize$ pointers are needed

Open addressing

- The collision is resolved by finding an available table entry other than the position (address) to which the colliding key is originally hashed.
- The simplest method is *linear probing*

Linear probing

- $(h(K) + i) \bmod TSize$.
- *If it is free, store there*
- *Otherwise*
 - *Try $A[h(k)+i]$*
 - *where $i=1,2,3,4\dots$ until an empty slot is found*
 - *If the end of the table is reached and no empty cell has been found, the search is continued from the beginning of the table*
 - *stops—in the extreme case—in the cell preceding the one from which the search started.*

Search (Linear probing)

- ▣ To search for key k
- ▣ *Look at the key stored at $A[\text{hash}(k)]$*
- ▣ *else try $A[\text{hash}(k)+1]$, $A[\text{hash}(k)+2]$, Until a key found or an empty slot is found*
 - If the end of the table is reached and no empty cell has been found, the search is continued from the beginning of the table
 - stops—in the extreme case—in the cell preceding the one from which the search started.

Remove (Linear probing)

- ▣ To remove for key k
- ▣ *If the key stored at $A[\text{hash}(k)]$ is k , remove it.*
- ▣ *else try $A[\text{hash}(k)+1]$, $A[\text{hash}(k)+2]$, Until a key found or an empty slot is found*
 - If the end of the table is reached and no empty cell has been found, the search is continued from the beginning of the table
 - stops—in the extreme case—in the cell preceding the one from which the search started.

Example

	Empty Table	After 89
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		89

□ $\text{hash}(x) = x \bmod 10$

□ $= 89 \bmod 10 = 9$

Continuous ...

□ $\text{hash}(x) = x \bmod 10$

□ $= 18 \bmod 10 = 8$

	Empty Table	After 89	After 18
0			
1			
2			
3			
4			
5			
6			
7			
8			18
9		89	89

Continuous ...

	Empty Table	After 89	After 18	After 49
0				49
1				
2				
3				
4				
5				
6				
7				
8			18	18
9		89	89	89

- $\text{hash}(x) = x \bmod 10$
- $= 49 \bmod 10 = 9$ //collision
- $\text{Probing} = (\text{hash} + 1) \% 10 = (9 + 1) \% 10 = 0$

Continuous ...

	Empty Table	After 89	After 18	After 49	After 58
0				49	49
1					58
2					
3					
4					
5					
6					
7					
8			18	18	18
9		89	89	89	89

- $\text{hash}(x) = x \bmod 10 = 58 \bmod 10 = 8$ //collision
- $\text{Probing} = (\text{hash} + 1) \% 10 = (8 + 1) \% 10 = 9$ //collision
- $= (8 + 2) \% 10 = 0$ //collision
- $= (8 + 3) \% 10 = 1$

Continuous ...

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1					58	58
2						69
3						
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

- $\text{hash}(x) = x \bmod 10 = 69 \bmod 10 = 9$ //collision
- $\text{Probing} = (\text{hash} + 1) \% 10 = (9 + 1) \% 10 = 0$ //collision
- $= (9 + 2) \% 10 = 1$ //collision
- $= (9 + 3) \% 10 = 2$

Problem with linear probing

- Insert 42,82,72,52, 32,12 in hash table.
- $h(\text{key}) = \text{key} \bmod 10$ //hash function

		42	82	72	52	32	12		
0	1	2	3	4	5	6	7	8	9

- `remove(int 72) //key=72`

		42	82		52	32	12		
0	1	2	3	4	5	6	7	8	9

- Search key=32
- Our search will stop after index 3 because we have found an empty space but 32 is exists in this array.
- We have break the chain by using this way.

Solution

- Use state removed previously
- At here, this state is shown by □

		42	82	72	52	32	12		
0	1	2	3	4	5	6	7	8	9

- `remove(int 72) //key=72`

		42	82	□	52	32	12		
0	1	2	3	4	5	6	7	8	9

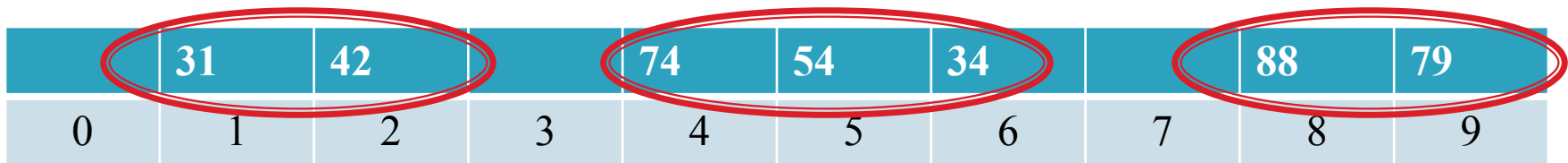
- Search `key=32`
- Now search will not stop at index 4 because it is not an empty cell rather it is previously used.

Three states for each location

- If an item is placed in $array[hash(key)+4]$, the item just before it is deleted.
- How will probe determine that the “hole” does not indicate the item is not in the array?
- three states for each location as:
 - Occupied
 - Empty (never used)
 - Deleted (previously used)

Problem Linear Probing

- This effect, known as **primary clustering**, means that **any key that hashes into the cluster** will require several attempts to resolve the collision, and then it will add to the cluster.
- Example
 - Three clusters are shown here by red circle.



A hash table with 10 slots, indexed 0 to 9. The slots contain the following values: 0 is empty, 1 contains 31, 2 contains 42, 3 is empty, 4 contains 74, 5 contains 54, 6 contains 34, 7 is empty, 8 contains 88, and 9 contains 79. Three clusters are highlighted with red circles: the first cluster contains slots 1 and 2 (values 31 and 42), the second cluster contains slots 4, 5, and 6 (values 74, 54, and 34), and the third cluster contains slots 8 and 9 (values 88 and 79). The empty slots at indices 0, 3, and 7 represent gaps between the clusters.

	31	42		74	54	34		88	79
0	1	2	3	4	5	6	7	8	9

Quadratic Hashing

- Quadratic probing is a collision resolution method that eliminates the primary clustering problem of linear probing.
- Hash Function with probing
 - $(h(K) + i^2) \bmod TSize$.
- *Normal function $h(k)$*
- *Prob i^2 such that $i=0,1,2,3,4....$*

Example

	Empty Table	After 89
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		89

□ $\text{hash}(x) = x \bmod 10$

□ $= 89 \bmod 10 = 9$

Continuous ...

□ $\text{hash}(x) = x \bmod 10$

□ $= 18 \bmod 10 = 8$

	Empty Table	After 89	After 18
0			
1			
2			
3			
4			
5			
6			
7			
8			18
9		89	89

Continuous ...

	Empty Table	After 89	After 18	After 49
0				49
1				
2				
3				
4				
5				
6				
7				
8			18	18
9		89	89	89

- $\text{hash}(x) = x \bmod 10$
- $= 49 \bmod 10 = 9$ //collision
- $\text{Probing} = (\text{hash} + I^2) \% 10 = (9 + 1) \% 10 = 0$

Continuous ...

	Empty Table	After 89	After 18	After 49	After 58
0				49	49
1					
2					58
3					
4					
5					
6					
7					
8			18	18	18
9		89	89	89	89

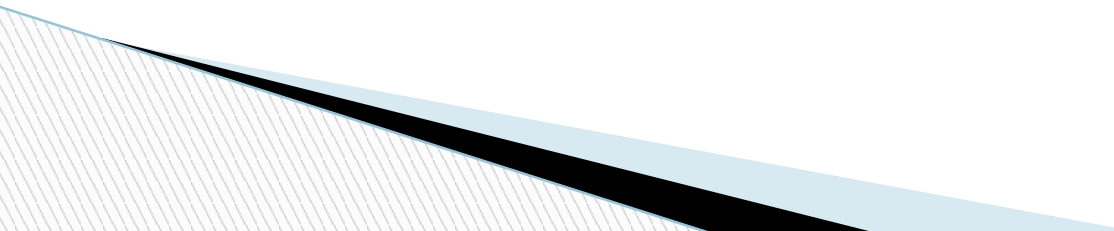
- $\text{hash}(x) = x \bmod 10 = 58 \bmod 10 = 8$ //collision
- $\text{Probing} = (\text{hash} + i^2) \% 10 = (8 + 1) \% 10 = 9$ //collision
- $= (8 + 4) \% 10 = 2$

Continuous ...

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1						
2					58	58
3						69
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

- $\text{hash}(x) = x \bmod 10 = 69 \bmod 10 = 9$ //collision
- $\text{Probing} = (\text{hash} + 1) \% 10 = (9 + 1) \% 10 = 0$ //collision
- $= (9 + 4) \% 10 = 3$

Probing Problems

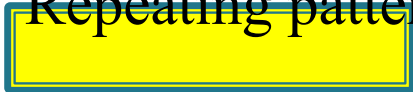
- For linear probing,
 - it is a bad idea to let the hash table get nearly full, because performance degrades.
 - For quadratic probing,
 - The situation is even more drastic:
 - There is no guarantee of finding an empty cell once the table gets more than half full, or even before the table gets half full if the table size is not prime.
 - This is because at most half of the table can be used as alternative locations to resolve collisions.
- 

Quadratic Hash Function Fail

- Hash Function with probing
 - $(h(K) + i^2) \bmod TSize$.
 - Where $h(k) = \text{key} \bmod 10$ and $Tsize=10$

60	71			54	85	94			89
0	1	2	3	4	5	6	7	8	9

- Let suppose we have already filled array with these values.
- Now we do not have the empty cell for key value 35
- Quadratic function will return these indices.
- 5,6,9,4,1,0,1,4,9,6,5,6,9,4,1,0,1,4,9,6,5,6,9.....
- Repeating pattern and function fail



□ Theorem 5.1 (Mark Allen Weiss)

- If quadratic probing is used, and the table size is prime, then a new element can always be inserted if the table is at least half empty.

Double hashing

- The last collision resolution method we will examine is **double hashing**.
- **Let we take these two hash functions.**
- $\text{hash}_1(x) = x \bmod 10$ //table size 10
- Function for second hashing
 - $f(i) = i \cdot \text{hash}_2(x)$ where i for probing i.e. $i = 1, 2, 3, \dots$
 - $\text{hash}_2(x) = R - (x \bmod R)$ such that R is prime number smaller than table size.
 - Let we choose $R = 7$

Example

	Empty Table	After 89
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		89

- $\text{hash}_1(x) = x \bmod 10$
- $= 89 \bmod 10 = 9$

Continuous ...

- $\text{hash}_1(x) = x \bmod 10$
- $= 18 \bmod 10 = 8$

	Empty Table	After 89	After 18
0			
1			
2			
3			
4			
5			
6			
7			
8			18
9		89	89

Continuous ...

	Empty Table	After 89	After 18	After 49
0				
1				
2				
3				
4				
5				
6				49
7				
8			18	18
9		89	89	89

- $\text{hash}_1(x) = x \bmod 10$
- $= 49 \bmod 10 = 9$
- Collision occur
- We choose $R = 7$ for this example
- $\text{hash}_2(x) = R - (x \bmod R) = 7 - (49 \bmod 7) = 7 - 0 = 7$
- 7 away from hash_1 value
i.e $(9+7) \bmod 10 = 6$

	Empty Table	After 89	After 18	After 49	After 58	After 69
0						69
1						
2						
3					58	58
4						
5						
6				49	49	49
7						
8			18	18	18	18
9		89	89	89	89	89

- $\text{hash}_1(x) = x \bmod 10$
- $= 69 \bmod 10 = 9$ //Collision occur

- $\text{hash}_2(x) = R - (x \bmod R) = 7 - (69 \bmod 7) = 7 - 6 = 1$
- 1 away from hash_1 value i.e $(9+1) \bmod 10 = 0$

	Empty Table	After 89	After 18	After 49	After 58	After 69
0						69
1						
2						
3					58	58
4						
5						
6				49	49	49
7						
8			18	18	18	18
9		89	89	89	89	89

- $\text{hash}_1(x) = x \bmod 10$
- $= 60 \bmod 10 = 0$ // **Collision occur**

- $\text{hash}_2(x) = R - (x \bmod R) = 7 - (60 \bmod 7) = 7 - 4 = 3$
- 3 away from hash_1 value i.e $(0+3) \bmod 10 = 3$ // **again collision**
- $i=2$ so $2 * \text{hash}_2(x) = 2 * 3 = 6$
- 6 away i.e $(0+6) \bmod 10 = 6$ // **again collision**
- For $i=3$ so $3 * \text{hash}_2(x) = 3 * 3 = 9$
- 9 away i.e $(0+9) \bmod 10 = 9$ // **again collision**
- For $i=4$ so $4 * \text{hash}_2(x) = 4 * 3 = 12$
- 12 away i.e $(0+12) \bmod 10 = 2$ // **finally inserted**

	Empty Table	After 89	After 18	After 49	After 58
0					
1					
2					
3					58
4					
5					
6				49	49
7					
8			18	18	18
9		89	89	89	89

- $\text{hash}_1(x) = x \bmod 10$
- $= 58 \bmod 10 = 8$ //Collision occur

- $\text{hash}_2(x) = R - (x \bmod R) = 7 - (58 \bmod 7) = 7 - 2 = 5$
- 7 away from hash_1 value i.e $(8+5) \bmod 10 = 3$

Rehashing

- If the table gets too full, the running time for the operations will start taking too long and insertions might fail for open addressing hashing with quadratic resolution.
- A solution, then, is to build another table that is about twice as big

Example

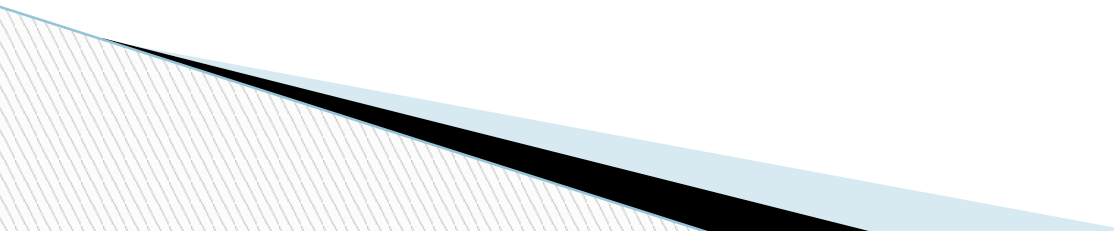
□ hash function is $h(x) = x \bmod 7$.

0	6
1	15
2	
3	24
4	
5	
6	13

23 is inserted

0	6
1	15
2	23
3	24
4	
5	
6	13

If 23 is inserted into the table, the resulting table in right side will be over 70 percent full.

- ❑ Create a new hash table double in size.
 - ❑ Scan the original hash table, compute new hash value and insert into the new hash table.
 - ❑ Free the memory occupied by the original hash table.
- 

Solution rehashing

0	
1	
2	
3	
4	
5	
6	6
7	23
8	24
9	
10	
11	
12	
13	13
14	
15	15
16	

- Because the table is so full, **a new table is created.**
- It is very expensive operation the running time is $O(N)$
- Table size is roughly $2N$
- The size of this table is 17
- This is the first prime that is twice as large as the old table size.
- The new hash function is then $h(x) = x \bmod 17$.